



第三章 高级功能(上海-悠悠)

第三章 高级功能(上海-悠悠)

3.1 new_context上下文与新窗口操作

- 3.1.1 浏览器上下文
- 3.1.2 Context实现测试浏览器环境隔离
- 3.1.3 多标签页

3.2 page.goto(url) 导航生命周期

- 3.2.1 导航生命周期
- 3.2.2 HTML DOM 事件
- 3.2.3 事件状态
- 3.2.4 使用示例
- 3.2.5 等待元素
- 3.2.6 timeout 等待超时

3.3 expect_popup() 处理新标签窗口

- 3.3.1 context.expect_page() 处理新标签页
- 3.3.2 处理弹出窗口

3.4 页面交互wait_for_load_state

- 3.4.1 自定义等待
- 3.4.2 显示等待
- 3.4.3 多重导航
- 3.4.4 加载弹出窗口
- 3.4.5 timeout 等待超时

3.5 强大的录制视频功能

- 3.5.1 录制视频
- 3.5.2 指定视频宽高
- 3.5.3 获取保存视频路径
- 3.5.4 使用示例

3.6 强大的Trace Viewer 测试追踪功能

- 3.6.1 Trace Viewer 追踪功能
- 3.6.2 查看
- 3.6.3 使用示例
- 3.6.4 查看追踪

3.7 保存cookie, 解决反复登录的问题

- 3.7.1 登录认证
- 3.7.2 storageState保存登录cookies
- 3.7.3 高级场景
- 3.8 强大的Mock 接口返回，模拟各种异常场景
 - 3.8.1 模拟网络请求
 - 3.8.2 修改 API
 - 3.8.3 模拟登录时候，服务器异常的场景
- 3.9 模拟手机浏览器测试
 - 3.9.1 设置手机模式
 - 3.9.2 playwright.devices 可支持的手机列表
 - 3.9.3 pytest-playwright 测试用例
- 3.10 启动Google Chrome 或 Microsoft Edge浏览器
 - 3.10.1 命令行下载
 - 3.10.2 指定 channel 打开浏览器
- 3.11 launch_persistent_context 启动本地 Google Chrome
 - 3.11.1 selenium 加载 Google Chrome 插件
 - 3.11.2 playwright 启动本地 Google Chrome
 - 3.11.3 配置args 参数加载插件
 - 3.11.4 使用代理

▪[网易云视频课程](#)

3.1 new_context上下文与新窗口操作

`browser.new_context()` 创建一个新的浏览器上下文。它不会与其他浏览器上下文共享 cookies/缓存。

3.1.1 浏览器上下文

使用`browser.new_context()` 创建context对象，context之间是相互隔离的，可以理解为轻量级的浏览器实例。

如需要不同用户登录同一个网页，不需要创建多个浏览器实例，只需要创建多个context即可

以下是在一个浏览器实例上打开2个标签页

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False, slow_mo=1000)
    context = browser.new_context() # 创建上下文, 浏览器实例
    page = context.new_page()      # 打开标签页1
    page.goto("https://www.baidu.com/")
    page2 = context.new_page()     # 打开标签页2
    page2.goto("https://www.cnblogs.com/yoyoketang/")

    context.close()
    browser.close()
```

也可以通过context 上下文操作多个浏览器实例, 它不会与其他浏览器上下文共享 cookies/缓存, 适用于多用户同时登陆的场景。

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False, slow_mo=1000)
    context1 = browser.new_context() # 创建上下文, 浏览器实例1
    context2 = browser.new_context() # 创建上下文, 浏览器实例2
    page1 = context1.new_page()      # 打开标签页1
    page1.goto("https://www.baidu.com/")

    # 操作第二个浏览器窗口
    page2 = context2.new_page()     # 打开标签页1
    page2.goto("https://www.baidu.com/")
```

运行后会发现打开了2个浏览器窗口

3.1.2 Context实现测试浏览器环境隔离

使用 Playwright 编写的测试在称为浏览器上下文的隔离的全新环境中执行。这种隔离模型提高了可重复性并防止级联测试失败。

什么是测试隔离

测试隔离是指每个测试与另一个测试完全隔离。每个测试都独立于任何其他测试运行。这意味着每个测试都有自己的本地存储、会话存储、cookie 等。Playwright 使用BrowserContext实现了这一点, 这相当于隐身式配置文件。它们的创建速度快、成本低, 并且完全隔离, 即使在单个浏

览器中运行也是如此。Playwright 为每个测试创建一个上下文，并在该上下文中提供一个默认页面。

为什么测试隔离很重要

- 没有失败结转。如果一个测试失败，它不会影响另一个测试。
- 易于调试错误或不稳定，因为您可以根据需要多次运行单个测试。
- 并行运行、分片等时不必考虑顺序。

测试隔离有两种不同的策略：从头开始或在两者之间进行清理。在测试之间清理的问题是很容易忘记清理，有些东西是不可能清理的，比如“访问过的链接”。来自一个测试的状态可能会泄漏到下一个测试中，这可能会导致您的测试失败并使调试变得更加困难，因为问题来自另一个测试。从头开始意味着一切都是新的，因此如果测试失败，您只需查看该测试即可进行调试。

Playwright 如何实现测试隔离

Playwright 使用浏览器上下文来实现测试隔离。每个测试都有自己的浏览器上下文。每次运行测试都会创建一个新的浏览器上下文。使用 Playwright 作为测试运行程序时，默认情况下会创建浏览器上下文。否则，您可以手动创建浏览器上下文。

```
browser = playwright.chromium.launch()
context = browser.new_context()
page = context.new_page()
```

浏览器上下文还可用于模拟涉及移动设备、权限、区域设置和配色方案的多页面场景

Playwright 可以在一个场景中创建多个浏览器上下文。当您想测试多用户功能（如聊天）时，这很有用。

```
from playwright.sync_api import sync_playwright

def run(playwright):
    # create a chromium browser instance
    chromium = playwright.chromium
    browser = chromium.launch()

    # create two isolated browser contexts
    user_context = browser.new_context()
    admin_context = browser.new_context()

    # create pages and interact with contexts independently

    with sync_playwright() as playwright:
        run(playwright)
```

3.1.3 多标签页

每个浏览器上下文可以承载多个页面（选项卡）。

- 每个页面都像一个聚焦的活动页面。不需要将页面置于最前面。
- 上下文中的页面遵循上下文级别的模拟，例如视口大小、自定义网络路由或浏览器区域设置。

```
# create two pages
page_one = context.new_page()
page_two = context.new_page()

# get pages of a browser context
all_pages = context.pages
```

使用示例，在page_one 标签页打开百度，输入“上海-悠悠”， 在page_two 标签页打开百度，输入“yoyoketang”

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False, slow_mo=1000)
    context = browser.new_context() # 创建上下文, 浏览器实例1

    page1 = context.new_page() # 打开标签页1
    page2 = context.new_page() # 打开标签页2
    page1.goto("https://www.baidu.com/")
    page1.fill('#kw', "上海-悠悠")
    page1.wait_for_timeout(3000)

    page2.goto("https://www.baidu.com/")
    page2.fill('#kw', "yoyoketang")
    page2.wait_for_timeout(3000)
```

虽然你看不到第一个页面的操作，实际上它已经操作了，每个页面 page 对象都是聚焦的活动页面，不需要将页面置于最前面。

3.2 page.goto(url) 导航生命周期

Playwright 可以导航到 URL 并处理由页面交互引起的导航。本篇涵盖了等待页面导航和加载完成的常见场景。

3.2.1 导航生命周期

导航从更改页面 URL 或通过与页面交互（例如，单击链接）开始。导航意图可能会被取消，例如，在点击未解析的 DNS 地址或转换为文件下载时。

解析响应标头并更新会话历史记录后，将提交导航。只有在导航成功（提交）后，页面才会开始加载文档。

加载包括通过网络获取剩余的响应主体、解析、执行脚本和触发加载事件：

调用 page.goto(url) 后页面加载过程：

- page.url 设定新的 url
- document 文档内容通过网络加载并解析
- page.on("domcontentloaded") 事件触发
- 执行页面的 js 脚本，页面执行一些脚本并加载 css 和图像等资源

- `page.on("load")` 事件触发
- 页面执行动态加载的脚本
- `networkidle` 当 500 毫秒内没有新的网络请求时触发

3.2.2 HTML DOM 事件

HTML DOM 事件允许Javascript在HTML文档元素中注册不同事件处理程序。

事件名称	作用
onload	通常用于 <code><body></code> 元素，在页面完全载入后(包括图片、css文件等等。)执行脚本代码。
onunload	用户退出页面。(<code><body></code> 和 <code><frameset></code>)
onclick	当用户点击某个对象时调用的事件句柄。
onmouseover	鼠标移到某元素之上。
onmouseout	鼠标从某元素移开。
ondblclick	当用户双击某个对象时调用的事件句柄。
onkeydown	某个键盘按键被按下。
onkeypress	某个键盘按键被按下并松开。
onkeyup	某个键盘按键被松开。
onfocus	元素获取焦点时触发
onblur	元素失去焦点时触发
onchange	该事件在表单元素的内容改变时触发 <code><input></code> , <code><keygen></code> , <code><select></code> , 和 <code><textarea></code>
onfocus	元素获取焦点时触发
onsubmit	表单提交时触发

3.2.3 事件状态

导航到 URL 会自动等待页面触发事件 `load` 。如果页面之前进行了客户端重定向 `load` , `page.goto()`将自动等待重定向页面触发事件load。

从源码可以看到 `wait_until` 等待的事件可以支持`["commit", "domcontentloaded", "load", "networkidle"]` 四个参数，默认是等待 `load` 触发。

`wait_until:Union["commit", "domcontentloaded", "load", "networkidle", None]`当认为操作成功时，默认为“加载”。事件可以是：

- `commit`：考虑在接收到网络响应并且文档开始加载时完成操作。
- `domcontentloaded`：在触发“domcontentloaded”事件时完成操作。
- `load`：触发 `load`事件时完成操作。
- `networkidle`：当 500 毫秒内没有新的网络请求时触发，认为操作已完成。

```
def goto(
    self,
    url: str,
    *,
    timeout: typing.Optional[float] = None,
    wait_until: typing.Optional[
        Literal["commit", "domcontentloaded", "load", "networkidle"]
    ] = None,
    referer: typing.Optional[str] = None
) -> typing.Optional["Response"]:
```

如果我们希望ajax 也请求完成了，再继续下一步，那么可以覆盖默认行为以等待特定事件，例如 `networkidle`。

(对于 `click`、`fill` 等操作会自动等待元素出现。)

使用语法

```
# Navigate and wait until network is idle
page.goto("https://example.com", wait_until="networkidle")
```

3.2.4 使用示例

当我们打开一个国外的网站，访问会很慢时（有些js/css资源加载很慢很慢）


```

from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False, slow_mo=1000)
    context = browser.new_context(accept_downloads=True)
    page = context.new_page()
    page.goto("https://playwright.dev/")

    page.get_by_role("link", name="Community").click()

    page.pause()
    browser.close()

```

如果是按默认的等待 load 事件，会出现报错 `waiting until "load"`

```

    result = next(iter(done)).result()
playwright._impl._api_types.TimeoutError: Timeout 30000ms exceeded.
===== logs =====
navigating to "https://playwright.dev/", waiting until "load"
=====

```

其实页面元素在 `domcontentloaded` 阶段就已经加载到元素了，那么不用等待到load

```

from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False, slow_mo=1000)
    context = browser.new_context(accept_downloads=True)
    page = context.new_page()
    page.goto("https://playwright.dev/", wait_until="domcontentloaded")

    page.get_by_role("link", name="Community").click()

    page.pause()
    browser.close()

```

对于有些元素可能需要等待到 `networkidle` 状态才会出来，那么就需要使用

```

page.goto("xxx", wait_until="networkidle")

```

3.2.5 等待元素

在延迟加载的页面中，使用locator.wait_for()等待元素可见是很有用的。或者，像page.click()这样的页面交互会自动等待元素。

```
# Navigate and wait for element
page.goto("https://example.com")
page.get_by_text("example domain").wait_for()

# Navigate and click element
# Click will auto-wait for the element
page.goto("https://example.com")
page.get_by_text("example domain").click()
```

3.2.6 timeout 等待超时

timeout 参数可以设置页面加载超时时间, 默认是30秒, 传递“0”以禁用超时。

```
timeout : Union[float, None]
    Maximum operation time in milliseconds, defaults to 30 seconds, pass `0` to disable timeout.
    The timeout can be changed by using the `browser_context.set_default_navigation_timeout()`,
    `browser_context.set_default_timeout()`, `page.set_default_navigation_timeout()` or
    `page.set_default_timeout()` methods.
```

更改默认值可以使用 以下方法更改

- browser_context.set_fault_navigation_timeout () 进行更改,
- browser_context.set_fault_timeout () ,
- page.set_fault_navigation_timeout)
- page.set_fault_timeout ()

3.3 expect_popup() 处理新标签窗口

当我们点击一个带有 target="_blank" 的链接时，会打开一个新的标签页，如果想继续在新的标签页上继续操作，那么有2种方式捕获到新页面的page对象

- 1. context.expect_page() 获取新标签页对象
- 2. page.expect_popup() 获取新标签页对象

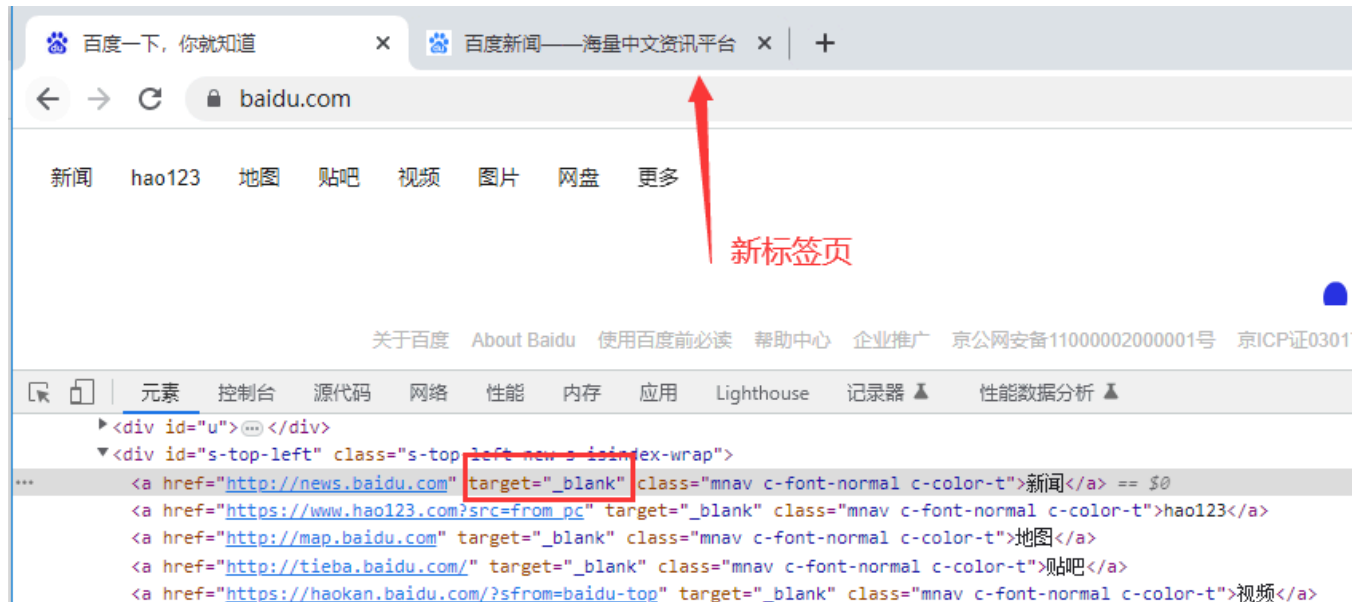
3.3.1 context.expect_page() 处理新标签页

浏览器上下文中的事件page可用于获取在上下文中创建的新页面。这可用于处理通过 `target="_blank"` 链接打开的新页面。

```
# Get page after a specific action (e.g. clicking a link)
with context.expect_page() as new_page_info:
    page.get_by_text("open new tab").click() # Opens a new tab
new_page = new_page_info.value

new_page.wait_for_load_state()
print(new_page.title())
```

使用示例，打开百度页面的-新闻链接，会出现一个新标签页



```

from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as p:
    browser = p.chromium.launch(headless=False, slow_mo=1000)
    context = browser.new_context() # 创建上下文, 浏览器实例

    page = context.new_page() # 打开标签页
    page.goto("https://www.baidu.com/")
    print(page.title())
    # Get page after a specific action (e.g. clicking a link)
    with context.expect_page() as new_page_info:
        page.click('text=新闻') # Opens a new tab
    new_page = new_page_info.value

    new_page.wait_for_load_state() # 等待页面加载到指定状态
    print(new_page.title())

```

运行结果

百度一下, 你就知道
 百度新闻—海量中文资讯平台

监听 context 的 page 事件

```

# Get all new pages (including popups) in the context
def handle_page(page):
    page.wait_for_load_state()
    print(page.title())

context.on("page", handle_page)

```

3.3.2 处理弹出窗口

如果页面打开一个弹出窗口（例如通过链接打开的页面），您可以通过监听页面上的事件 `target="_blank"` 来获取对它的引用。popup

除了 `browserContext.on('page')` 事件之外还会发出此事件，但仅针对与此页面相关的弹出窗口。

```
# Get popup after a specific action (e.g., click)
with page.expect_popup() as popup_info:
    page.get_by_text("open the popup").click()
    popup = popup_info.value

popup.wait_for_load_state()
print(popup.title())
```

监听page的popup事件

```
# Get all popups when they open
def handle_popup(popup):
    popup.wait_for_load_state()
    print(popup.title())

page.on("popup", handle_popup)
```

3.4 页面交互wait_for_load_state

从A页面点击打开B页面，存在2种情况

- 1.带有 `target="_blank"` 的链接时，会打开一个新的标签页
- 2.直接在当前页面刷新另外一个地址

当打开一个新的页面后，可以使用 `wait_for_load_state` 等待页面加载到指定状态
等待的事件可以支持["commit", "domcontentloaded", "load", "networkidle"] 四个参数

3.4.1 自定义等待

`locator.click`可以结合`page.wait_for_load_state()`来等待加载事件。

```
page.locator("button").click() # Click triggers navigation
page.wait_for_load_state("networkidle") # This waits for the "networkidle"
```

3.4.2 显示等待

显式调用`page.expect_navigation()` 等待新的页面加载

如果没有等待到新的导航页面，就会报超时异常。

```
# Waits for the next navigation. Using Python context manager
# prevents a race condition between clicking and waiting for a navigation.
with page.expect_navigation():
    # Triggers a navigation after a timeout
    page.get_by_text("Navigate after timeout").click()
```

3.4.3 多重导航

点击一个元素可能会触发多个导航。在这些情况下，建议将 `page.expect_navigation()` 显式指向特定的 url。

```
# Using Python context manager prevents a race condition
# between clicking and waiting for a navigation.
with page.expect_navigation(url="**/login"):
    # Triggers a navigation with a script redirect
    page.get_by_text("Click me").click()
```

3.4.4 加载弹出窗口

打开弹出窗口时，显式调用 `page.wait_for_load_state()` 可确保将弹出窗口加载到所需状态。

```
with page.expect_popup() as popup_info:
    page.get_by_text("Open popup").click() # Opens popup
popup = popup_info.value
popup.wait_for_load_state("load")
```

3.4.5 timeout 等待超时

`timeout` 参数可以设置页面加载超时时间，默认是30秒，传递“0”以禁用超时。

```
timeout : Union[float, None]
    Maximum operation time in milliseconds, defaults to 30 seconds, pass `0` to disable timeout. The default
    be changed by using the `browser_context.set_default_navigation_timeout()`,
    `browser_context.set_default_timeout()`, `page.set_default_navigation_timeout()` or
    `page.set_default_timeout()` methods.
```

更改默认值可以使用以下方法更改

- `browser_context.set_fault_navigation_timeout()` 进行更改,

- `browser_context.set_fault_timeout ()` ,
- `page.set_fault_navigation_timeout()`
- `page.set_fault_timeout ()`

3.5 强大的录制视频功能

使用 Playwright, 您可以为测试录制视频。

3.5.1 录制视频

视频在浏览器上下文关闭时保存。如果您手动创建浏览器上下文, 请确保 `browser_context.close()`, 会在调用close的时候保存视频。

```
context = browser.new_context(record_video_dir="videos/")

# 确保调用 close, videos视频才会保存
context.close()
```

执行完成后, 会保存到videos/目录下

名称	日期	类型	大小	时长
 652e10d397b491...	2023/2/27 13:40	WEBM 文件	11 KB	00:00:01
 532750b41682f48...	2023/2/27 13:42	WEBM 文件	471 KB	00:00:41

视频录制40秒左右, 也才400k大小, 一般用户3-5秒可以执行完, 所以完全不用担心视频占用太多的空间。

3.5.2 指定视频宽高

您还可以指定视频大小。视频大小默认为缩小以适合 800x800 的视口大小。

视口的视频放置在输出视频的左上角, 必要时按比例缩小以适合。您可能需要设置视口大小以匹配您想要的视频大小。

```
context = browser.new_context(  
    record_video_dir="videos/",  
    record_video_size={"width": 640, "height": 480}  
)
```

3.5.3 获取保存视频路径

保存的视频文件将出现在指定的文件夹中。它们都生成了唯一的名称。对于多页面场景，可以通过 `page.video` 访问页面关联的视频文件。

```
path = page.video.path()  
print(path) # videos\b880519a32528f80d64a2cb6769f2162.webm
```

3.5.4 使用示例

```
from playwright.sync_api import sync_playwright  
  
with sync_playwright() as p:  
    browser = p.chromium.launch(headless=False, slow_mo=1000)  
    context = browser.new_context(  
        record_video_dir="videos/",  
        record_video_size={"width": 640, "height": 480}  
    )  
    page = context.new_page()  
    page.goto("http://47.108.155.10/login.html")  
    page.get_by_placeholder("请输入用户名").fill("test")  
    page.get_by_placeholder("请输入密码").fill("123456")  
    page.get_by_role("button", name="立即登录 >").click()  
  
    path = page.video.path()  
    print(path) # videos\b880519a32528f80d64a2cb6769f2162.webm  
  
    # 确保调用 close, videos视频才会保存  
    context.close()  
    browser.close()
```

3.6 强大的Trace Viewer 测试追踪功能

在执行自动化用例的过程中，出现一些不稳定偶然性的bug，需要复现bug，还原bug出现的过程

程。于是需要追踪用例执行的过程。

Playwright Trace Viewer 是一个 GUI 工具，可让您探索记录的 Playwright 测试跟踪，这意味着您可以在测试的每个操作中前后移动，并直观地查看每个操作期间发生的情况。

3.6.1 Trace Viewer 追踪功能

可以使用 `browser_context.tracing` API 记录跟踪，如下所示：

```
browser = chromium.launch()
context = browser.new_context()

# Start tracing before creating / navigating a page.
context.tracing.start(screenshots=True, snapshots=True, sources=True)

page = context.new_page()
page.goto("https://playwright.dev")

# Stop tracing and export it into a zip archive.
context.tracing.stop(path = "trace.zip")
```

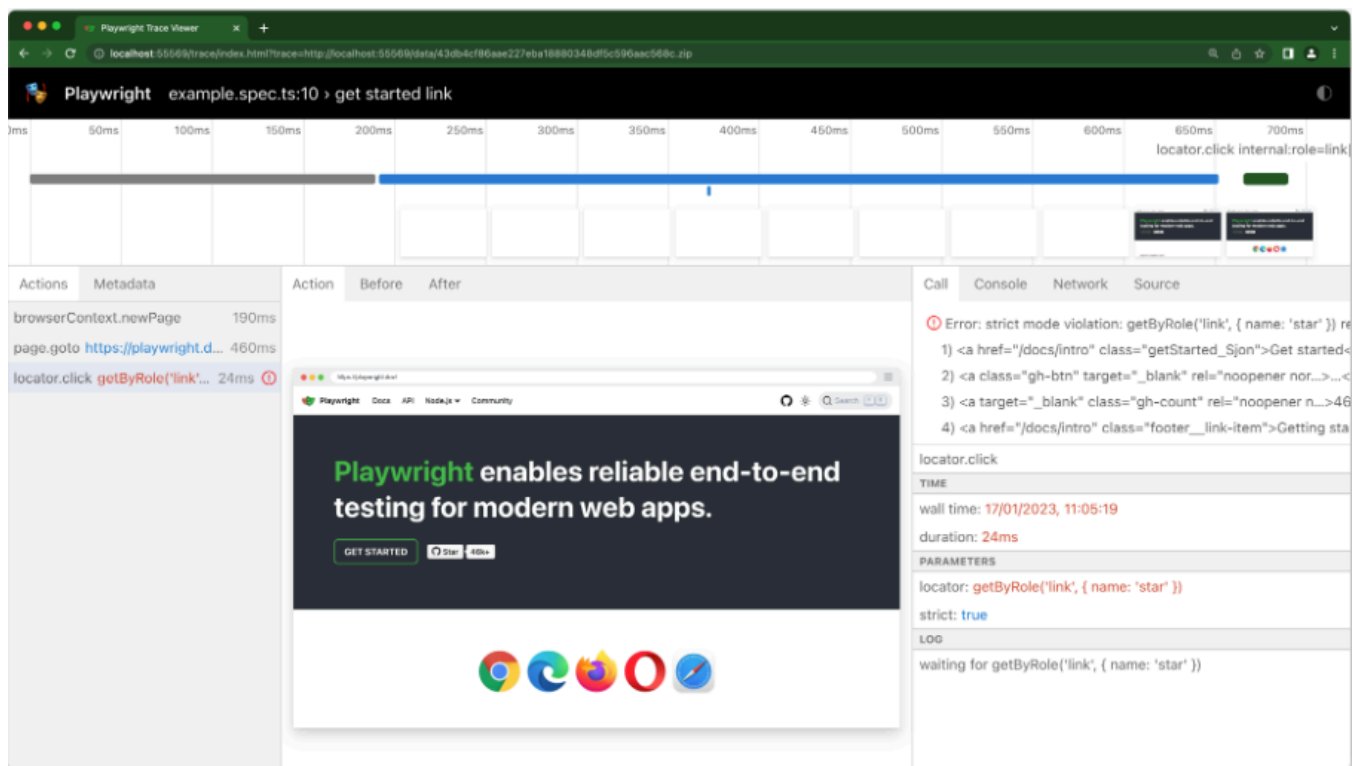
这将记录跟踪并将其放入名为 `trace.zip`。

您可以使用 Playwright CLI 或在您的浏览器中打开保存的跟踪 `trace.playwright.dev`。

```
playwright show-trace trace.zip
```

3.6.2 查看

通过单击每个操作或使用时间轴悬停来查看测试的痕迹，并查看操作前后页面的状态。在测试的每个步骤中检查日志、源和网络。跟踪查看器创建一个 DOM 快照，因此您可以与其完全交互，打开 devtools 等。



3.6.3 使用示例

打开网站登录，记录操作过程

```
from playwright.sync_api import Playwright, sync_playwright, expect
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

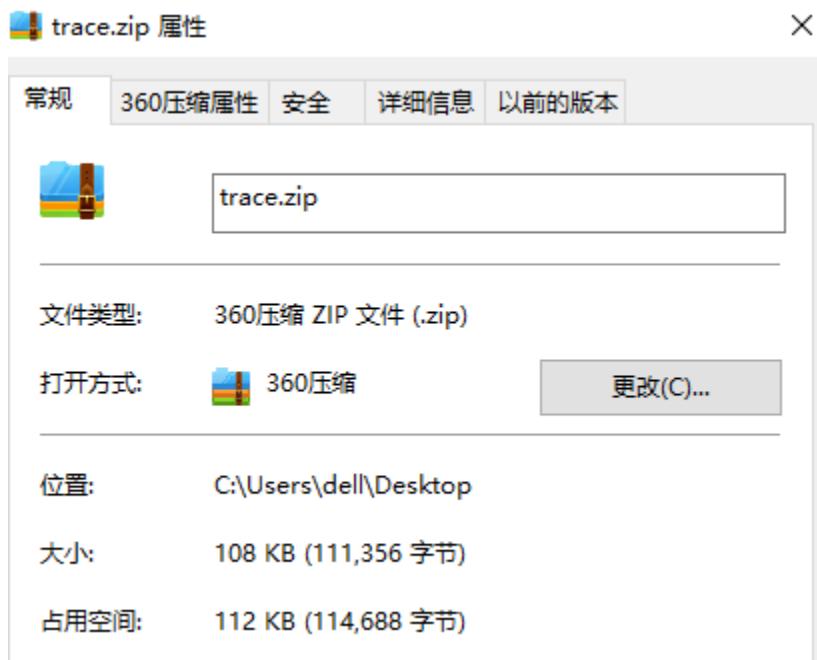
def run(playwright: Playwright) -> None:
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context()

    # Start tracing before creating / navigating a page.
    # 配置追踪开始
    context.tracing.start(screenshots=True, snapshots=True, sources=True)
    # 打开页面
    page = context.new_page()
    page.goto('http://127.0.0.1:8000/login.html')
    page.get_by_label("用户名:").fill("yoyo")
    page.get_by_label("密码:").fill("123456")
    page.locator("text=立即登录").click()

    context.tracing.stop(path="trace.zip") # 结束追踪的地方, 添加 tracing 的结束配置。
    context.close()
    browser.close()

with sync_playwright() as playwright:
    run(playwright)
```

运行结束后在本地保存一个 `trace.zip` 功能, 并且文件只有100K左右, 占用空间很小



3.6.4 查看追踪

有2种方法可以查看追踪日志

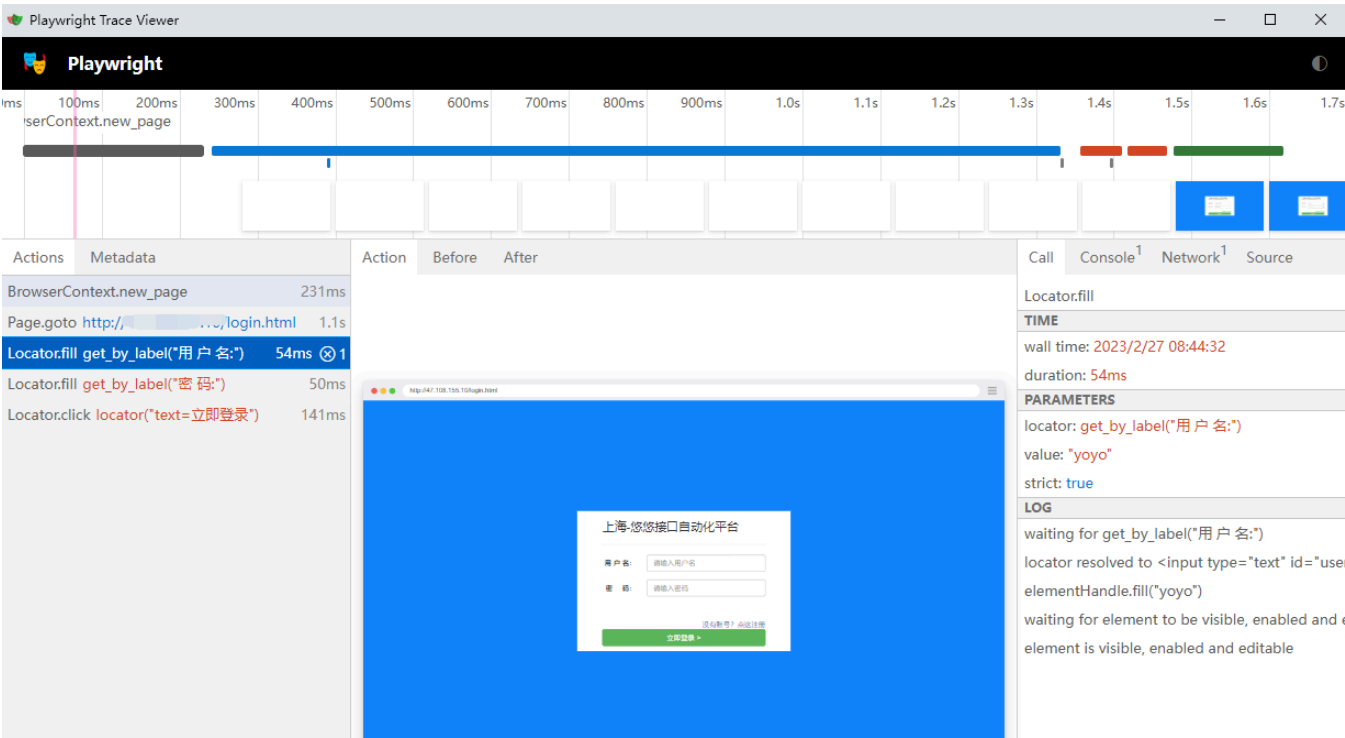
方法1. 通过命令行查看操作过程追踪

```
>playwright show-trace trace.zip
```

方法2.访问 <https://trace.playwright.dev/> 选择录制好的trace.zip文件即可打开



它可以清晰的记录代码的每个步骤



并且还能记录每个动作前和后的对比

Actions	Metadata	Action	Before	After	Call	Console ¹	Network ¹	Source
BrowserContext.new_page	231ms				Locator.fill			
Page.goto http://47.108.155.10/login.html	1.1s				TIME			
Locator.fill get_by_label("用户名:")	54ms				wall time: 2023/2/27 08:44:32			
Locator.fill get_by_label("密码:")	50ms				duration: 54ms			
Locator.click locator("text=立即登录")	141ms				PARAMETERS			
					locator: get_by_label("用户名:")			
					value: "yoyo"			
					strict: true			
					LOG			
					waiting for get_by_label("用户名:")			
					locator resolved to <input type="text" id="use			
					elementHandle.fill("yoyo")			
					waiting for element to be visible, enabled and			
					element is visible, enabled and editable			

输入用户名之前

Actions	Metadata	Action	Before	After	Call	Console ¹	Network ¹	Source
BrowserContext.new_page	231ms				Locator.fill			
Page.goto http://47.108.155.10/login.html	1.1s				TIME			
Locator.fill get_by_label("用户名:")	54ms				wall time: 2023/2/27 08:44:32			
Locator.fill get_by_label("密码:")	50ms				duration: 54ms			
Locator.click locator("text=立即登录")	141ms				PARAMETERS			
					locator: get_by_label("用户名:")			
					value: "yoyo"			
					strict: true			
					LOG			
					waiting for get_by_label("用户名:")			
					locator resolved to <input type="text" id="use			
					elementHandle.fill("yoyo")			
					waiting for element to be visible, enabled and			
					element is visible, enabled and editable			

输入之后

Playwright 的 Trace Viewer 功能非常强大，执行过程中出现的问题可以快速定位

3.7 保存cookie，解决反复登录的问题

在写web自动化的时候，很多用例是需要先登录的，为了做到每个用例的隔离，数据互不影响，一般会创建一个新的page对象。

很多用例是需要先登录的，可以先登录后把cookie保存到本地，通过加载cookie的方式解决重复登录的问题。

3.7.1 登录认证

Playwright 在称为浏览器上下文的隔离环境中执行测试。这种隔离模型提高了可重复性并防止级联测试失败。

测试可以加载现有的经过身份验证的状态。这消除了在每个测试中进行身份验证的需要并加快了测试执行速度。

我们建议创建 `auth` 目录并将其添加到您的 `.gitignore`。您的身份验证例程将生成经过身份验证的浏览器状态并将其保存到此 `auth` 目录中的文件中。稍后，测试将重用此状态并开始已通过身份验证。

windows 操作命令

```
md auth
echo. >> .gitignore
echo "auth" >> .gitignore
```

以下示例登录到 GitHub。执行这些步骤后，浏览器上下文将通过身份验证。

```
page = context.new_page()
page.goto('https://github.com/login')

# Interact with login form
page.get_by_label("Username or email address").fill("username")
page.get_by_label("Password").fill("password")
page.get_by_role("button", name="Sign in").click()
# Continue with the test
```

为每个测试重做登录会减慢测试执行速度。为了缓解这种情况，请改用现有的身份验证状态

3.7.2 storageState保存登录cookies

Playwright 提供了一种在测试中重用登录状态的方法。这样您就可以只登录一次，然后跳过所有测试的登录步骤。

Web 应用程序使用基于 cookie 或基于令牌的身份验证，其中经过身份验证的状态存储为 cookie 或本地存储。Playwright 提供 `browserContext.storageState([options])` 方法，可用于从经过身份验证的上下文中检索存储状态，然后创建具有预填充状态的新上下文。

Cookie 和本地存储状态可以跨不同的浏览器使用。它们取决于您的应用程序的身份验证模型：某些应用程序可能需要 cookie 和本地存储。

以下代码片段从经过身份验证的上下文中检索状态，并使用该状态创建一个新上下文。

```

from playwright.sync_api import Playwright, sync_playwright, expect
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

def run(playwright: Playwright) -> None:
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context()
    page = context.new_page()
    page.goto('https://github.com/login')

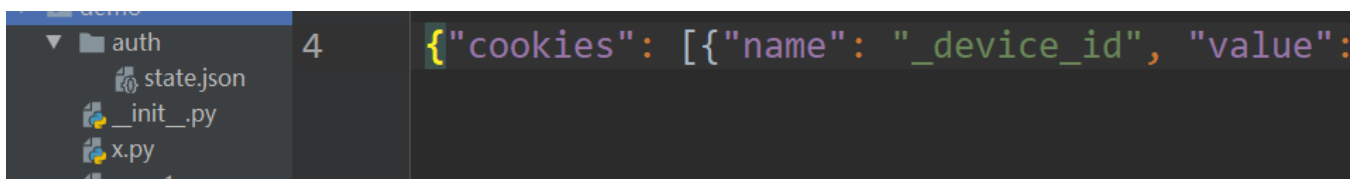
    # Interact with login form
    page.get_by_label("Username or email address").fill("yoyo")
    page.get_by_label("Password").fill("*****")
    page.get_by_role("button", name="Sign in").click()
    # Continue with the test

    # 保存storage state 到指定的文件
    storage = context.storage_state(path="auth/state.json")

    # -----
    context.close()
    browser.close()

with sync_playwright() as playwright:
    run(playwright)

```



于是在本地会保存一个state.json文件

这样在其它地方就可以使用本地的cookies，实现免登录了

```

# Create a new context with the saved storage state.
context = browser.new_context(storage_state="state.json")

```

验证下是不是已经不需要登录了

```
from playwright.sync_api import Playwright, sync_playwright, expect
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

def run(playwright: Playwright) -> None:
    browser = playwright.chromium.launch(headless=False)
    # 加载本地cookies, 免登陆
    context = browser.new_context(storage_state="auth/state.json")

    # 打开页面继续操作
    page = context.new_page()
    page.goto('https://github.com/')
    page.pause() # 打断点看是不是已经登录了

    context.close()
    browser.close()

with sync_playwright() as playwright:
    run(playwright)
```

3.7.3 高级场景

重用经过身份验证的状态包括cookie和基于本地存储的身份验证。会话存储很少用于存储与登录状态相关的信息。会话存储特定于特定域，并且不会跨页面加载持续存在。Playwright 不提供持久会话存储的 API，但以下代码片段可用于保存/加载会话存储。

使用添加到环境变量的方式


```
import os
# Get session storage and store as env variable
session_storage = page.evaluate("() => JSON.stringify(sessionStorage)")
os.environ["SESSION_STORAGE"] = session_storage

# Set session storage in a new context
session_storage = os.environ["SESSION_STORAGE"]
context.add_init_script("""(storage => {
  if (window.location.hostname === 'example.com') {
    const entries = JSON.parse(storage)
    for (const [key, value] of Object.entries(entries)) {
      window.sessionStorage.setItem(key, value)
    }
  }
}))(session_storage + session_storage + '')")
```

3.8 强大的Mock 接口返回， 模拟各种异常场景

web 自动化主要测前端UI 的功能， 有很多异常的场景， 我们很难造真实的场景去触发， 比如服务器异常时候， 前端的提示语。

这时候就可以使用mock 功能， 模拟接口的返回， 测试前端的功能。

3.8.1 模拟网络请求

Web API 通常作为 HTTP 端点实现。Playwright 提供 API 来模拟和修改网络流量， 包括 HTTP 和 HTTPS。页面执行的任何请求， 包括XHR和获取请求， 都可以被跟踪、修改和模拟。

以下代码将拦截对的所有调用https://dog.ceo/api/breeds/list/all并将返回测试数据。不会向https://dog.ceo/api/breeds/list/all端点发出任何请求。

```
async def handle(route):
    json = { message: { "test_breed": [] } }
    route.fulfill(json=json)

page.route("https://dog.ceo/api/breeds/list/all", handle)
```

3.8.2 修改 API

有时， 必须发出 API 请求， 但需要修补响应以允许可重现的测试。在这种情况下， 可以执行请求并使用修改后的响应来完成请求， 而不是模拟请求。

```
def handle(route):
    response = route.fulfill()
    json = response.json()
    json["message"]["big_red_dog"] = []
    # Fulfill using the original response, while patching the response body
    # with the given JSON object.
    route.fulfill(response=response, json=json)

page.route("https://dog.ceo/api/breeds/list/all", handle)
```

3.8.3 模拟登录时候，服务器异常的场景

当登录的接口返回状态码是500 的时候，前端才会触发：服务器异常！

于是可以用 `page.route()` 拦截请求，修改状态码即可触发

```

from playwright.sync_api import Playwright, sync_playwright, expect
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

def handle(route):
    # 状态码改成500 模拟服务器异常
    route.fulfill(status=500)

def run(playwright: Playwright) -> None:
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context()
    page = context.new_page()
    page.goto("http://127.0.0.0:8000/login.html")
    page.get_by_placeholder("请输入用户名").click()
    page.get_by_placeholder("请输入用户名").fill("yoyo")
    page.get_by_placeholder("请输入密码").click()
    page.get_by_placeholder("请输入密码").fill("aa123456")

    page.route("http://47.108.155.10/api/login", handle)

    page.get_by_role("button", name="立即登录 >").click()
    page.pause() # 断点
    # -----
    context.close()
    browser.close()

with sync_playwright() as playwright:
    run(playwright)

```

这就给我们测试前端的各种异常场景带来了很大的方便，可以模拟出任何我们希望返回的接口数据。这对于前后端分离的项目测试是非常关键的。

3.9 模拟手机浏览器测试

使用 Playwright，您可以在任何浏览器上测试您的应用程序，也可以模拟真实设备，例如手机或平板电脑。

3.9.1 设置手机模式

只需配置您想要模拟的设备，Playwright 就会模拟浏览器行为，例

如"user_agent"、"screen_size"以及"viewport"是否"has_touch"启用。
您还可以为所有测试或特定测试模拟,以及设置以"geolocation"显示通知或更改."locale""timezone""permissions""colorScheme"

Playwright使用 `playwright.devices` 为选定的台式机、平板电脑和移动设备提供设备参数注册表。

它可用于模拟特定设备的浏览器行为,例如用户代理、屏幕尺寸、视口以及是否启用了触摸。所有测试都将使用指定的设备参数运行。

使用示例

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog: https://www.cnblogs.com/yoyoketang/

def run(playwright):
    iphone_12 = playwright.devices['iPhone 12']
    browser = playwright.chromium.launch(headless=False)
    context = browser.new_context(
        **iphone_12,
    )
    page = context.new_page()
    page.goto('https://m.baidu.com')
    page.pause()

with sync_playwright() as playwright:
    run(playwright)
```

需注意手机设备参数必须是playwright 支持的设备列表里面的手机型号才可以。

运行后就是以手机模式打开的



3.9.2 playwright.devices 可支持的手机列表

有2种方式可以查看 playwright.devices 所支持的全部设置

第一种，直接打印 playwright.devices

```
with sync_playwright() as p:
    import pprint
    pprint.pprint(p.devices)
```

第二种，查看deviceDescriptorsSource.json源码

playwright可以支持的手机列表可以在GitHub 上查看到

<https://github.com/microsoft/playwright/blob/main/packages/playwright-core/src/server/deviceDescriptorsSource.json>

查看到 iPhone 12 手机相关配置参数

```
'iPhone 12': {'default_browser_type': 'webkit',
              'device_scale_factor': 3,
              'has_touch': True,
              'is_mobile': True,
              'user_agent': 'Mozilla/5.0 (iPhone; CPU iPhone OS 14_4 like Mac '
                            'OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) '
                            'Version/16.4 Mobile/15E148 Safari/604.1',
              'viewport': {'height': 664, 'width': 390}},
```

3.9.3 pytest-playwright 测试用例

在pytest-playwright 测试用例中可以通过重写browser_context_args 来实现

```
# conftest.py

import pytest
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

@pytest.fixture(scope="session")
def browser_context_args(browser_context_args, playwright):
    iphone_11 = playwright.devices['iPhone 11 Pro']
    return {
        **browser_context_args,
        **iphone_11,
    }
```

用例部分

```
from playwright.sync_api import Page

def test_m_baidu(page: Page):
    page.goto("https://m.baidu.com/")
```

这样可以指定手机型号运行用例

3.10 启动Google Chrome 或 Microsoft Edge浏览器

playwright 默认会下载 chromium,firefox 和 webkit 三个浏览器，目前支持通过命令下载的浏览器有：chromium、chrome、chrome-beta、msedge、msedge-beta、msedge-dev、firefox、

webkit

3.10.1 命令行下载

使用 `playwright install` 命令默认会安装chromium,firefox 和 webkit 三个浏览器。

可以通过 `playwright -h` 命令查看目前支持的浏览器

```
>playwright install --help
Usage: playwright install [options] [browser...]

ensure browsers necessary for this version of Playwright are installed

Options:
  --with-deps  install system dependencies for browsers
  --dry-run    do not execute installation, only print information
  --force      force reinstall of stable browser channels
  -h, --help   display help for command

Examples:
  - $ install
    Install default browsers.

  - $ install chrome firefox
    Install custom browsers, supports chromium, chrome, chrome-beta, msedge, msedge-beta, msedge-dev, firefox, webkit
```

从命令行帮助信息中可以看到支持的浏览器有:chromium, chrome, chrome-beta, msedge, msedge-beta, msedge-dev, firefox, webkit

安装指定的浏览器, 如果本机已经安装过了, 就不会安装了

```
playwright install chrome

playwright install msedge
```

Google Chrome 或 Microsoft Edge 安装不会被隔离。它们将安装在默认的全局位置, 具体取决于您的操作系统。

如果提示已经在系统里面安装了chrome

Failed to install browsers

Error:

```
|| ATTENTION: "chrome" is already installed on the system! ||
||
|| "chrome" installation is not hermetic; installing newer version ||
|| requires *removal* of a current installation first. ||
||
|| To *uninstall* current version and re-install latest "chrome": ||
||
|| - Close all running instances of "chrome", if any ||
|| - Use "--force" to install browser: ||
||
||     playwright install --force chrome ||
||
|| <3 Playwright Team ||
```

可以先关闭正在运行的chrome 浏览器，使用以下命令安装到最新版

```
playwright install --force chrome
```

安装完成会显示版本号，以及安装的位置

```
>playwright install --force chrome
Downloading Google Chrome
Installing Google Chrome

ProductVersion  FileVersion      FileName
-----
111.0.5563.65   111.0.5563.65    C:\Program Files\Google\Chrome\Application\chrome.exe
```

3.10.2 指定 channel 打开浏览器

默认情况下，chromium.launch() 不带 channel 参数打开的是 `chromium` 浏览器


```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as pw:
    browser = pw.chromium.launch(headless=False)
    page = browser.new_page()

    page.goto("https://www.baidu.com/")
```

Google Chrome 和 Microsoft Edge浏览器都是用的 chromium 内核，所以只需加一个 `channel="chrome"` 即可打开谷歌浏览器

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as pw:
    browser = pw.chromium.launch(headless=False, channel="chrome")
    page = browser.new_page()

    page.goto("https://www.baidu.com/")
```

添加 `channel="msedge"` 即可打开Microsoft Edge浏览器

```
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

with sync_playwright() as pw:
    browser = pw.chromium.launch(headless=False, channel="msedge")
    page = browser.new_page()

    page.goto("https://www.baidu.com/")
```

3.11 launch_persistent_context 启动本地 Google Chrome

playwright 启动Google Chrome 浏览器的时候默认用的是无痕模式，不加载本地的数据，这对于测试人员运行一个干净的浏览器是没问题的。

大家在学selenium的时候，知道 selenium 可以启动本地的 Google Chrome 浏览器并加载本地数据，这样可以在本地已经登录过网站的情况下，下次打开网站不需要登录继续操作，对于一些爬虫用户是非常重要的功能。

playwright 可以使用launch_persistent_context方法启动本地的chrome 浏览器

3.11.1 selenium 加载 Google Chrome 插件

在启动浏览器的时候添加 `--user-data-dir` 用户数据目录，即可启动带插件的浏览器，并且会记住用户的cookies数据

```
from selenium import webdriver
import getpass
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

# 启用带插件的浏览器
option = webdriver.ChromeOptions()
option.add_argument(f"--user-data-dir=C:\\Users\\{getpass.getuser()}\\AppData\\Local\\Google\\Chrome\\User Data")
driver = webdriver.Chrome(chrome_options=option) # 打开chrome浏览器
driver.get("https://www.cnblogs.com/yoyoketang/")
```

3.11.2 playwright 启动本地 Google Chrome

使用 launch_persistent_context 方法启动本地的chrome 浏览器，并且设置 `channel="chrome"`

```

import getpass
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

# 获取 google chrome 的本地缓存文件
USER_DIR_PATH = f"C:\\Users\\{getpass.getuser()}\\AppData\\Local\\Google\\Chrome\\User Data"

with sync_playwright() as p:
    browser = p.chromium.launch_persistent_context(
        # 指定本机用户缓存地址
        user_data_dir=USER_DIR_PATH,
        # 接收下载事件
        accept_downloads=True,
        # 设置 GUI 模式
        headless=False,
        bypass_csp=True,
        slow_mo=1000,
        channel="chrome",
    )

    page = browser.new_page()
    page.goto("https://www.cnblogs.com/yoyoketang")

    page.pause()

```

在运行的时候，先关闭本地的chrome 浏览器，再执行代码，就可以看到启动的浏览器，打开网站不需要登录了，但是插件并没有加载进去。

3.11.3 配置args 参数加载插件

Google Chrome 插件加载需配置args 参数，所有的 args 参数列表可以在这个地址查询 <https://peter.sh/experiments/chromium-command-line-switches/>

<code>--disable-dev-shm-usage</code> ^[19]	The /dev/shm partition is too small in certain VM environments, causing Chrome to fail or crash (see http://crbug.com/715363). Use this flag to work-around this issue (a temporary directory will always be used to create anonymous shared memory files).
<code>--disable-device-disabling</code>	If this switch is set, the device cannot be remotely disabled by its owner.
<code>--disable-dinosaur-easter-egg</code>	Disables the dinosaur easter egg on the offline interstitial.
<code>--disable-domain-blocking-for-3d-apis</code>	Disable the per-domain blocking for 3D APIs after GPU reset. This switch is intended only for tests.
<code>--disable-domain-reliability</code>	Disables Domain Reliability Monitoring.
<code>--disable-explicit-dma-fences</code>	Disable explicit DMA-fences
<code>--disable-extensions</code>	Disable extensions.
<code>--disable-extensions-except</code>	Disable extensions except those specified in a comma-separated list.
<code>--disable-extensions-file-access-check</code>	Disable checking for user opt-in for extensions that want to inject script into file URLs (ie, always allow it). This is used during automated testing.
<code>--disable-extensions-http-throttling</code>	Disable the net::URLRequestThrottlerManager functionality for requests originating from extensions.
<code>--disable-features</code>	Comma-separated list of feature names to disable. See also kEnableFeatures.
<code>--disable-fetching-hints-at-navigation-start</code>	No description
<code>--disable-field-trial-config</code>	Disable field trial tests configured in fieldtrial_testing_config.json.
<code>--disable-file-system</code>	Disable FileSystem API.
<code>--disable-fine-grained-time-zone-detection</code>	Disables fine grained time zone detection.

在使用脚本加载扩展插件时，一定要解压crx文件，不要直接安装crx

```

import getpass
from playwright.sync_api import sync_playwright
# 上海悠悠 wx:283340479
# blog:https://www.cnblogs.com/yoyoketang/

# 获取 google chrome 的本地缓存文件
USER_DIR_PATH = f"C:\\Users\\{getpass.getuser()}\\AppData\\Local\\Google\\Chrome\\User Data"
# chrome.exe指定可执行文件路径
# executable_path = "chromium-111/chrome-win/chrome.exe"

# chrome插件目录, 下载后解压crx
path_to_extension = f"\\Users\\Downloads\\extension\\xxx"

with sync_playwright() as p:
    browser = p.chromium.launch_persistent_context(
        # 指定本机用户缓存地址
        user_data_dir=USER_DIR_PATH,
        # executable_path=executable_path, # 如果有需要可以下载chrome.exe到指定目录加载
        # 接收下载事件
        accept_downloads=True,
        # 设置 GUI 模式
        headless=False,
        bypass_csp=True,
        slow_mo=1000,
        channel="chrome",
        args=[
            f"--disable-extensions-except={path_to_extension}",
            f"--load-extension={path_to_extension}"
        ], # 加载扩展插件
    )

```

扩展程序仅在 Chrome / Chromium GUI模式中使用。

3.11.4 使用代理

使用代理可以在launch_persistent_context 加上proxy 参数

```

proxy=ProxySettings(server="http://xxx.xxx.xxx.xxx:xxxx"),

```

网易云视频课程

网易云视频完整课程地址<https://study.163.com/course/courseMain.htm?courseId=1213382811&share=2&shareId=480000002230338>

